CIC – 209: DATA STRUCTURES

Unit – I

Arrays

Arrays

- An array is defined as a set of finite number of homogeneous elements or same data items.
- It means an array can contain one type of data only, either all integer, all float-point number or all character.
- The individual elements are typically stored in consecutive memory locations.
- The length of the array is determined when the array is created, and cannot be changed.
- Any component of the array can be inspected or updated by using its index.
  - *This is an efficient operation*
  - *O(1) = constant time*

Arrays

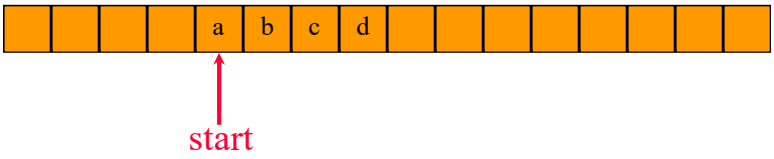
- Some common operation performed on array are:
  - Creation of an array
  - Traversing an array
  - Insertion of new element
  - Deletion of required element
  - Modification of an element
  - Sorting of elements of an array
  - Merging of arrays
  - Searching for an element in the array

Different Types of Arrays

- **One-dimensional array:** only one index is used
- **Multi-dimensional array:** array involving more than one index
- **Static array:** the compiler determines how memory will be allocated for the array
- **Dynamic array:** memory allocation takes place during execution

1-D Array Representation in C/C++

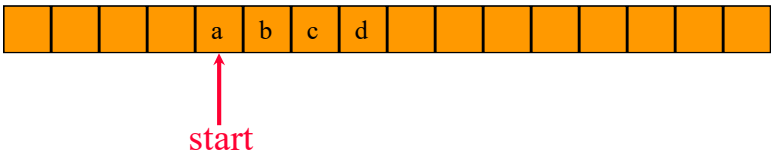
Memory



- 1-dimensional array x = [a, b, c, d]
- map into contiguous memory locations
- location(x[i]) = start + i (i.e. x+i)

Space Overhead

Memory



Space Overhead = 4 bytes for start (i.e pointer pointing to the array)

(excludes space needed for the elements of x)

2-D Arrays

The elements of a 2-dimensional array a declared as:

int a[3][4];

may be shown as a table

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

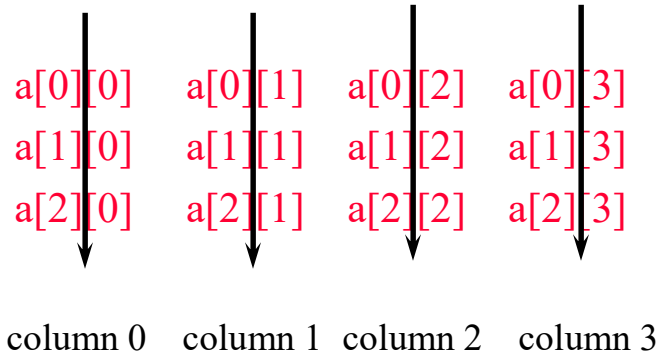
Rows Of A 2D Array

a[0][0] a[0][1] a[0][2] a[0][3] → row 0

a[1][0] a[1][1] a[1][2] a[1][3] → row 1

a[2][0] a[2][1] a[2][2] a[2][3] → row 2

Columns Of A 2D Array



2-D Array Representation in C/C++

2-dimensional array x

a, b, c, d

e, f, g, h

i, j, k, l

view 2-D array as a 1-D array of rows

x = [row0, row1, row 2]

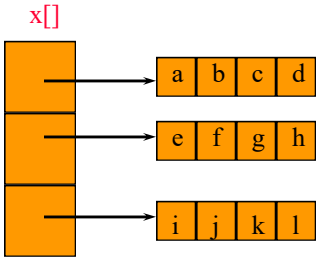
row 0 = [a,b, c, d]

row 1 = [e, f, g, h]

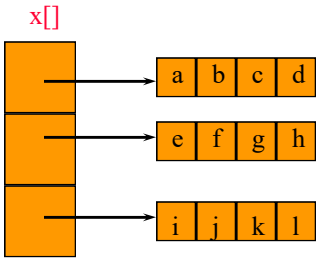
row 2 = [i, j, k, l]

and store as 4 1-D arrays

2-D Array Representation in C/C++

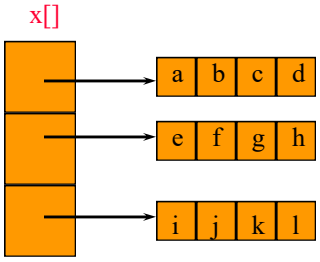


Space Overhead



Space Overhead = Overhead for 4 1-D arrays
= 4 * 4 bytes
= 16 bytes
= (Number of Rows + 1) x 4 bytes

Array Representation in C/C++



- This representation is called the Array-of-Arrays representation.
- Requires contiguous memory of size 3, 4, 4, and 4 for the 4 1-D arrays.
- 1 memory block of size Number of Rows and Number of Rows blocks of size Number of Columns

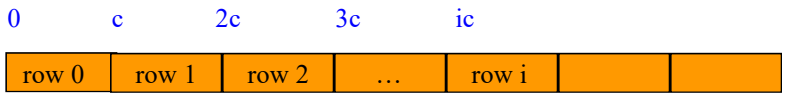
Row-Major Order (RMO) Mapping

- Consider 2-D array x[3][4]:

a b c d
e f g h
i j k l

- Convert into 1-D array y by collecting elements by rows.
- Within a row elements are collected from left to right.
- Rows are collected from top to bottom.
- We get 1-D array y[] = {a, b, c, d, e, f, g, h, i, j, k, l}

Locating Element x[i][j] in RMO



- assume array x has r rows and c columns
- each row has c elements
- i rows to the left of i^{th} row
- so ic elements to the left of x[i][0]
- so x[i][j] is mapped to position

ic + j of the 1-D array

or (i x no_of_columns + j)

Column-Major Order (CMO) Mapping

- Consider 2-D array x[3][4]:

a b c d

e f g h

i j k l

- Convert into 1-D array y by collecting elements by columns.
- Within a column elements are collected from top to bottom.
- Columns are collected from left to right.
- We get 1-D array y[] = {a, e, i, b, f, j, c, g, k, d, h, l}

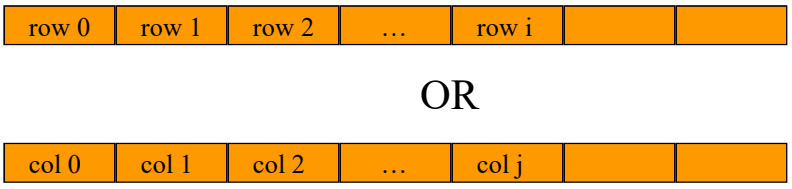
Locating Element x[i][j] in CMO

- assume array x has r rows and c columns
- each column has r elements
- j columns to the left of j^{th} column
- so jr elements to the left of x[0][j]
- so x[i][j] is mapped to position

  i + jr of the 1-D array

  or (i + j x no_of_rows)

Space Overhead



4 bytes for start of 1-D array +

4 bytes {either for c (number of columns) or

for r (number of rows)

= 8 bytes

Disadvantage

Need contiguous memory of size rc.

Matrix

Table of values. Has rows and columns, but numbering begins at 1 rather than 0.

a b c d row 1

e f g h row 2

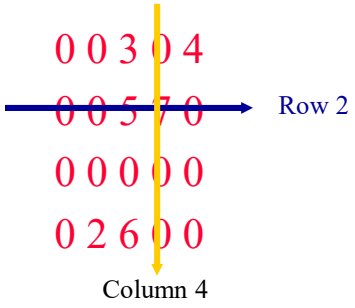
i j k l row 3

- Use notation x(i,j) rather than x[i][j].
- May use a 2-D array to represent a matrix.

Sparse Matrices

Matrix➔ table of values

Sparse matrix ➔ #nonzero elements/#elements is small.



4 x 5 matrix

4 rows

5 columns

20 elements

6 nonzero elements

Representation Of Sparse Matrices

- Single linear list in Row-Major Order.
- Scan the nonzero elements of the sparse matrix in row-major order (i.e., scan the rows left to right beginning with row 1 and picking up the nonzero elements)
- Each nonzero element is represented by a triple or 3-tupple

  (row, column, value)
- The list of triples is stored in a 1-D array

Single Linear List Example

0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0

list =

row	0	0	1	1	3	3
column	2	4	2	3	1	2
value	3	4	5	7	2	6

One Linear List Per Row

0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0

row0 = [(2, 3), (4,4)]
row1 = [(2,5), (3,7)]
row2 = []
row3 = [(1,2), (2,6)]

Approximate Memory Requirements

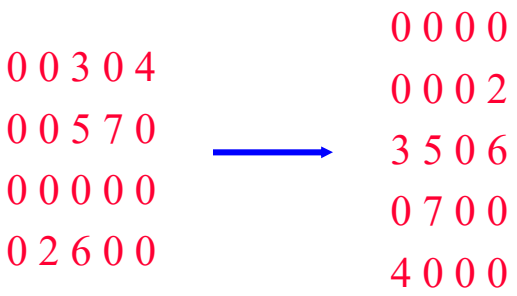
500 x 500 matrix with 1994 nonzero elements, 4 bytes per element

2-D array = 500 x 500 x 4 = 1 million bytes

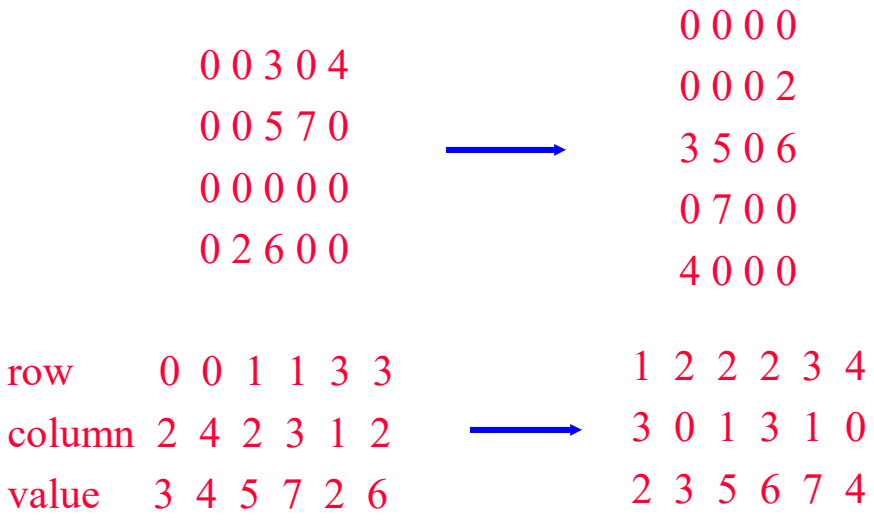
3-tupple Representation = (3 x 1994 x 4) + 4 x 4

= 23,944 bytes

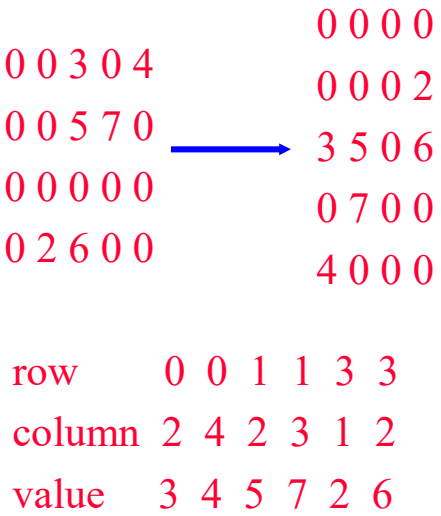
Matrix Transpose



Matrix Transpose



Matrix Transpose



Step 1: #nonzero in each row of transpose.
= #nonzero in each column of original matrix
= [0, 1, 3, 1, 1]

Step2: Start of each row of transpose
= sum of size of preceding rows of transpose
= [0, 0, 1, 4, 5]

Step 3: Move elements, left to right, from original list to transpose list.

Matrix Transpose

Step 1: #nonzero in each row of transpose.

= #nonzero in each column of original matrix

= [0, 1, 3, 1, 1]

Step2: Start of each row of transpose

= sum of size of preceding rows of transpose

= [0, 0, 1, 4, 5]

Step 3: Move elements, left to right, from original list to transpose list.

Complexity

m x n original matrix

t nonzero elements

Step 1: O(n+t)

Step 2: O(n)

Step 3: O(t)

Overall O(n+t)